

# Planetario 3D

## Relazione del progetto

Applicazione interattiva in C++ con SFML, OpenGL e GLM

Modello divulgativo del sistema solare sviluppato in dodici tappe, con mesh, texture, illuminazione, orbite, stelle e camera interattiva.

Studente:  
Corso:  
Repository:

Manuel Venarotta  
Fondamenti di Computer Grafica  
<https://github.com/Manuvena/Planetario-3D>

## 1 Obiettivo e impostazione del progetto

Il progetto che ho realizzato è un planetario 3D interattivo. La scena finale contiene il sole, gli otto pianeti, la luna, le rispettive orbite e lo sfondo stellato. Con il mouse posso ruotare la camera, mentre con la tastiera posso mettere in pausa la simulazione e modificarne la velocità.

Non ho riprodotto una simulazione astronomica esatta, ma ho scelto di rappresentare il sistema solare usando un modello divulgativo pensato per rendere contemporaneamente visibili tutti i corpi celesti e mettere in pratica gli elementi affrontati nel corso.

### 1.1 Gestione delle scale

Le dimensioni reali dei pianeti sono troppo piccole rispetto alle distanze che li separano. Inoltre, le distanze aumentano molto andando verso i pianeti esterni. Usando un'unica scala realistica avrei ottenuto pianeti quasi invisibili e una scena molto difficile da osservare.

Ho quindi deciso di mantenere l'ordine dei pianeti con differenze di dimensione riconoscibili, ingrandendo leggermente i corpi più piccoli e riducendo la distanza tra le orbite. Anche le velocità non sono quelle reali, sono state scelte e accelerate per rendere meglio visibili rivoluzione e rotazione. Nella tappa finale ho anche invertito il verso della rotazione propria di Venere e Urano. Le inclinazioni degli assi non sono state rappresentate.

### 1.2 Tecnologie e organizzazione

Il programma è scritto in C++. SFML gestisce la finestra, il contesto OpenGL, le immagini e gli eventi, OpenGL si occupa del disegno della scena, GLAD carica le funzioni OpenGL, GLM e i file di supporto del laboratorio gestiscono vettori, matrici, mesh e shader. CMake permette di compilare tutte le tappe con un unico comando.

Il repository contiene le cartelle **Tappa01** fino a **Tappa12**, una cartella comune **Risorse**, gli header in **include** e il file **CMakeLists.txt**. Gli eseguibili vengono creati in **build/bin**, le risorse vengono copiate in **build/Risorse**.

## 2 Sviluppo del progetto per tappe

**Tappa 01 - finestra SFML e ciclo di rendering.** Ho iniziato con una finestra `sf::RenderWindow` e un cerchio verde disegnato tramite SFML. Questa prima tappa mi è servita per verificare la compilazione e mostrare la gestione degli eventi, la pulizia del back buffer, il disegno e la chiamata a `display`.

**Tappa 02 - contesto OpenGL.** Sono passato da `RenderWindow` a `sf::Window` e ho richiesto un contesto OpenGL. Ho caricato le funzioni con GLAD, attivato il depth test e pulito sia il buffer colore sia quello di profondità.

**Tappa 03 - primo triangolo sulla GPU.** Ho definito tre vertici in CPU, li ho copiati in un VBO e ho configurato un VAO per spiegare alla GPU come leggerli. Il triangolo viene disegnato con `glDrawArrays`. Ho aggiunto anche l'aggiornamento del viewport quando la finestra cambia dimensione. Questa tappa è il primo collegamento completo tra dati C++, shader e chiamata di disegno.

**Tappa 04 - mesh OFF, camera e classi principali.** Ho sostituito il triangolo con un cubo caricato dal file `cubo.off`. Ho separato il programma nelle classi `Setup`, `Camera`, `GPUMesh` e `Scene`. `GPUMesh` carica la mesh indicizzata e crea VBO, VAO ed EBO. In questa tappa lo shader usa la posizione dei vertici. La camera costruisce le matrici di view e projection e può essere ruotata trascinando il mouse. La model matrix centra, ridimensiona e anima il cubo.

**Tappa 05 - gerarchia sole, terra e luna.** Ho riutilizzato la stessa mesh cubica tre volte per creare una prima struttura simile a un sistema solare. La terra viene ruotata attorno al sole e la luna viene costruita partendo dal sistema di riferimento della terra. La parte importante di questa tappa è la gerarchia: quando si muove il centro della terra, anche la luna lo segue.

**Tappa 06 - colori distinti.** Fino alla tappa precedente tutti gli oggetti usavano lo stesso colore. Ho aggiunto l'uniform `color_obj` nel fragment shader e, prima di ogni chiamata di disegno, invio un colore diverso: giallo per il sole, azzurro per la terra e grigio per la luna.

Già in queste tappe la stessa geometria viene riutilizzata più volte. Non creo una mesh diversa per ogni corpo: cambio la model matrix e gli uniform prima di richiamare `mesh.draw()`. Questa scelta rimane alla base della versione finale.

**Tappa 07 - sfera generata dal programma e illuminazione.** Ho introdotto la funzione `generaSferaOFF`, che costruisce una sfera tramite paralleli e meridiani e salva vertici e facce triangolari in un file di formato `.OFF` (`sfera.off`). Il VAO ora descrive sia la posizione sia la normale. Nel fragment shader ho aggiunto una leggera luce ambientale e una luce diffusa calcolata rispetto alla sorgente posta nel centro del sole. Il sole viene escluso da questo calcolo e mostra direttamente il proprio colore.

**Tappa 08 - texture dei corpi celesti.** Ho caricato le immagini con `sf::Image` e creato per ciascuna una texture OpenGL. Successivamente trasferisco i pixel in memoria GPU con `glTexImage2D`, genero le mipmap e imposto i filtri di minification e magnification. Nel fragment shader ricavo le coordinate della texture dalla posizione locale sulla sfera. Collegando un'immagine diversa, la stessa mesh assume l'aspetto del sole, della terra o della luna.

**Tappa 09 - tempo simulato e comandi.** Ho separato il tempo della simulazione dal numero di fotogrammi usando il tempo trascorso `dt`. Con la barra spaziatrice posso mettere in pausa l'aggiornamento, con le frecce modifico invece `time_scale`. Se il valore diventa negativo, i movimenti procedono nel verso opposto.

**Tappa 10 - orbite visibili.** Ho creato la classe `GPUOrbit`, che costruisce ogni orbita come una linea circolare nel piano  $xz$ . Il costruttore riceve il raggio e il numero di segmenti. La CPU calcola con seno e coseno i punti della circonferenza e ripete alla fine il primo punto, così la linea si chiude. Le coordinate vengono caricate nel VBO e nel VAO vengono salvate le istruzioni di lettura per la GPU. Nel metodo `draw()`, `glDrawArrays` con `GL_LINE_STRIP` collega i punti in ordine. Non servono EBO, facce, normali o texture, perché l'orbita è una linea e non una superficie. L'uniform `is_orbit` permette al fragment shader di riconoscerla e assegnarle il colore scelto senza applicare texture o illuminazione.

**Tappa 11 - cielo stellato e ampliamento del sistema.** Ho aggiunto la classe `GPUSStars` per creare lo sfondo stellato senza costruire una sfera per ogni stella. La CPU genera posizioni casuali usando `theta`, `phi` e `r`: `theta` distribuisce le stelle attorno alla scena, `phi` le posiziona ad altezze diverse e `r` le mantiene lontane dai pianeti. Ogni stella è un solo vertice 3D, le posizioni vengono caricate in un VBO e il VAO descrive come leggerle. Il metodo `draw()` usa `GL_POINTS`, quindi ogni vertice viene disegnato come un punto. L'uniform `is_star` permette al fragment shader di riconoscere le stelle e assegnare direttamente un colore chiaro. In questa tappa ho aggiunto mercurio, venere, marte e giovè, le loro texture e le rispettive orbite.

**Tappa 12 - versione completa.** Ho completato il sistema aggiungendo saturno, urano e nettuno. La scena finale usa otto orbite planetarie, l'orbita lunare, dieci texture e 1200 stelle. Ho aumentato la distanza della camera e il piano di taglio lontano per riuscire a visualizzare l'intero sistema. Stelle, orbite, sole e pianeti usano la stessa coppia di shader, ma vengono distinti tramite gli uniform `is_star`, `is_orbit` e `is_sun`.

### 3 Costruzione della sfera OFF

Costruisco la sfera dividendola in paralleli e meridiani. Per ogni parallelo genero tutti i meridiani e con seno e coseno trasformo gli angoli `phi` e `theta` nelle coordinate 3D dei vertici. I due poli vengono trattati separatamente, perché molti triangoli devono convergere nello stesso punto.

La parte centrale è composta da celle delimitate da due paralleli consecutivi e due meridiani consecutivi. Ogni cella ha quattro vertici, ma OpenGL lavora con triangoli, per questo la divido in due facce. Il modulo fa tornare l'indice dal meridiano finale al meridiano iniziale e impedisce che rimanga una fessura nella sfera.

```
for (int i=0; i<parallels-1; i++) {
    for (int j=0; j<meridians; j++) {
        int next_j=(j+1)%meridians;
        int v0=1+i*meridians+j;
        int v1=1+i*meridians+next_j;
        int v2=1+(i+1)*meridians+next_j;
        int v3=1+(i+1)*meridians+j;
        out<<"3 "<<v0<<" "<<v1<<" "<<v3<<"\n";
        out<<"3 "<<v1<<" "<<v2<<" "<<v3<<"\n";
    }
}
```

Con 40 paralleli e 40 meridiani ottengo 1602 vertici e 3200 triangoli. Il file dichiara prima queste quantità, poi contiene le coordinate e infine gli indici delle facce.

### 4 Da Mesh a GPUMesh

La classe `Mesh` legge il file OFF e ricostruisce vertici, facce e normali lato CPU. `pack4gpu` prende i dati del file OFF e prepara due vettori, uno conterrà i vertici con le informazioni su posizione e normale, l'altro gli indici dei vertici. Il VBO contiene i dati dei vertici, l'EBO contiene l'ordine con cui formare i triangoli e il VAO ricorda come devono essere letti i dati.

```
glVertexAttribPointer(0,3, GL_FLOAT, GL_FALSE, 6*sizeof(float), (void*)0);
glEnableVertexAttribArray(0); // posizione: primi tre float
glVertexAttribPointer(1,3, GL_FLOAT, GL_FALSE, 6*sizeof(float),
    (void*)(3*sizeof(float)));
glEnableVertexAttribArray(1); // normale: tre float successivi
glDrawElements(GL_TRIANGLES, index_count, GL_UNSIGNED_INT, 0);
```

Le location 0 e 1 corrispondono a `pos` e `norm` nel vertex shader. Grazie agli indici, gli stessi vertici possono essere usati da più triangoli senza duplicare tutte le coordinate.

## 5 Gerarchia terra-luna

La sfera letta dal file viene prima centrata nell'origine e ridimensionata tramite `mesh_base_norm`:

```
mesh_base_norm = scaling(1.0f/mesh.extent)
                * translation(-mesh.center);
```

La traslazione porta il centro della mesh nell'origine, mentre la scala divide le sue dimensioni per `mesh.extent`. Ottengo così una sfera base centrata e con una dimensione prevedibile. `mesh_base_norm` non posiziona ancora un corpo nella scena: prepara la stessa geometria, che poi posso riutilizzare cambiando la model matrix e la texture. Le trasformazioni sono applicate da destra verso sinistra: prima normalizzo e scalo la sfera, poi applico la rotazione propria, la allontano dal sole e infine ruoto quella distanza per ottenere la rivoluzione.

La matrice `c_ter` rappresenta il centro mobile della terra. La terra parte da quel riferimento e la luna usa lo stesso riferimento come genitore:

```
glm::mat4 c_ter = pos_sole
    * rotation_y(t*25.0f)
    * translation(4.6f,0.0f,0.0f);

glm::mat4 m_ter = c_ter * rotation_y(t*80.0f)
    * scaling(0.38f) * mesh_base_norm;

glm::mat4 m_luna = c_ter * rotation_y(t*110.0f)
    * translation(0.7f,0.0f,0.0f)
    * scaling(0.13f) * mesh_base_norm;
```

Se cambia la posizione della terra, cambia automaticamente anche l'origine dell'orbita lunare. La stessa idea di gerarchia viene usata per collegare i pianeti al sole: le loro model matrix partono da `pos_sole` e aggiungono poi rivoluzione, distanza, rotazione propria e scala. Nella simulazione il centro del sole rimane fermo nell'origine. Il sole ruota su se stesso, ma non riceve una traslazione orbitale e quindi non si sposta. I pianeti ruotano attorno a questo centro fisso, mentre la luna segue il centro mobile della terra.

## 6 Camera orbitante

La camera usata nel progetto è orbitante, perché osserva sempre il centro della scena e permette di girargli attorno. La variabile `phi` rappresenta l'angolo orizzontale e decide da quale lato osservi il sistema, `theta` rappresenta l'angolo verticale e decide se lo osservi più dall'alto o più dal basso, `od` rappresenta la distanza della camera dal centro della scena. Nella tappa finale `od` rimane fissa.

Il trascinamento orizzontale del mouse modifica `phi`, mentre quello verticale modifica `theta`. Quest'ultimo valore viene limitato tra -89 e 89 gradi per evitare che la camera si ribalti. La view matrix combina la traslazione lungo l'asse z, determinata da `od`, con le rotazioni date da `phi` e `theta`. La projection matrix è prospettica e tiene conto del rapporto tra larghezza e altezza della finestra e dei piani oltre i quali gli oggetti non vengono disegnati.

## 7 Texture, shader e pipeline di rendering

### 7.1 Caricamento e utilizzo delle texture

`caricaTexture` legge l'immagine in RAM. `glGenTextures` crea un ID OpenGL e lo salva in `tex_id`, che contiene il numero assegnato alla texture e serve a identificarla nella GPU. Dopo il bind, `glTexImage2D` trasferisce i pixel e le funzioni successive generano mipmap e filtri. Il ribaltamento verticale compensa la diversa origine usata dalle immagini e da OpenGL.

Tutte le texture vengono lette attraverso `GL_TEXTURE0`. L'uniform `tex_sampler` riceve il numero 0, quindi il fragment shader sa da dove leggere. Prima di disegnare un corpo eseguo il bind della sua texture. La mesh non cambia, cambia solamente la texture collegata alla chiamata di disegno.

### 7.2 Vertex shader e fragment shader

Il vertex shader riceve posizione e normale dal VAO. Trasforma la posizione locale nello spazio del mondo e poi nello spazio di clip. La normale viene trasformata con la matrice inversa trasposta, perché una direzione non deve ricevere la traslazione e deve restare corretta anche dopo una scalatura.

Il fragment shader segue tre percorsi. Le stelle ricevono direttamente un colore chiaro, le orbite ricevono `color_obj`, per i corpi sferici vengono calcolate le coordinate della texture. Il sole mostra direttamente la propria texture, mentre i pianeti e la luna ricevono illuminazione ambientale e diffusa.

```
vec3 n = -normalize(v_norm);
vec3 lightDir = normalize(light_pos-v_pos);
float diff = max(dot(n,lightDir),0.0);
vec3 ambient = 0.05*texColor.rgb;
vec3 diffuse = diff*texColor.rgb;
c = vec4(ambient+diffuse,1.0);
```

Il prodotto scalare misura quanto la superficie è rivolta verso il sole, `max` impedisce di illuminare il lato opposto con valori negativi. Nella mesh le normali risultano rivolte verso l'interno della sfera. Senza il segno meno, la luce comparirebbe sul lato opposto rispetto al sole. Con `-normalize(v_norm)` inverte quindi la direzione delle normali e le faccio puntare verso l'esterno, in modo che il prodotto scalare illumini il lato corretto.

### 7.3 Ordine di un fotogramma

La CPU gestisce eventi e tempo, aggiorna gli uniform e seleziona VAO e texture. Il vertex shader trasforma i vertici, l'assemblatore costruisce punti, linee o triangoli, la rasterizzazione produce i frammenti, il fragment shader calcola il colore, il depth test decide quali frammenti restano visibili. Infine SFML presenta il back buffer sullo schermo.

## 8 Interazione, problemi incontrati e soluzioni

### 8.1 Comandi della tappa finale

| Comando                        | Effetto                                                        |
|--------------------------------|----------------------------------------------------------------|
| Mouse sinistro e trascinamento | Ruota la camera attorno alla scena.                            |
| Barra spaziatrice              | Mette in pausa o riprende la simulazione.                      |
| Freccia su                     | Aumenta <code>time_scale</code> .                              |
| Freccia giù                    | Riduce <code>time_scale</code> , può anche invertire il tempo. |
| Ridimensionamento finestra     | Aggiorna viewport e aspect ratio.                              |

### 8.2 Problemi e soluzioni adottate

| Problema                | Causa                                                               | Soluzione                                                                                                   |
|-------------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Risorse non trovate     | Il percorso parte dalla cartella dell'eseguibile, non dal file C++. | Cartella comune <code>Risorse</code> , percorsi relativi <code>../Risorse/...</code> e copia tramite CMake. |
| Pianeti poco visibili   | Le scale astronomiche reali sono incompatibili con una vista unica. | Scale indipendenti e divulgative per dimensioni e distanze.                                                 |
| Fessura nella sfera     | L'ultimo meridiano deve essere collegato al primo.                  | Uso del modulo sugli indici e gestione separata dei poli.                                                   |
| Texture capovolte       | Immagini e coordinate OpenGL usano origini verticali differenti.    | Uso di <code>img.flipVertically()</code> e immagini piane preparate per essere applicate alla sfera.        |
| Luce sul lato sbagliato | Le normali della mesh sono rivolte verso l'interno.                 | Normalizzazione e inversione della normale prima del prodotto scalare.                                      |

## 9 Risorse esterne e uso dell'intelligenza artificiale

Le librerie SFML, OpenGL, GLAD e GLM sono quelle previste dal corso. La base di CMake e i file di supporto `hotshaders.hh`, `matrices.hh` e `mesh.hh` derivano dal materiale e dal template dei laboratori e sono stati riutilizzati e adattati nel progetto. Il file `cubo.off` è preso dal materiale delle esercitazioni sulle mesh. La sfera finale, invece, viene generata direttamente dal programma.

Mi sono aiutato con strumenti di intelligenza artificiale per rivedere alcune parti del codice, per la scrittura del codice usato per la generazione del file OFF della sfera e per capire il funzionamento e il codice del vertex shader e del fragment shader. Ho usato l'intelligenza artificiale anche per la parte di codice che riguarda il caricamento e l'uso delle texture, per generare le texture dei corpi celesti e per correggere la relazione.

## 10 Conclusioni

Il risultato finale rispetta l'obiettivo che mi ero posto: costruire un modello del sistema solare leggibile e interattivo, senza trasformarlo in una simulazione fisica troppo complessa. Le dodici tappe mostrano un percorso continuo: parto dal ciclo di rendering, arrivo alla gestione dei dati in GPU e integro poi mesh, trasformazioni gerarchiche, camera, texture, luce, orbite, stelle e input.